



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Faculty of Electrical and Computer Engineering

“Continuous Integration”

WORKSHOP - SECOND TERM

SOFTWARE ENGINEERING II

Group Members:

Adrián Fierro José Andrés
Romero Yagual Bruno Rafael
Otero Limones Alex David
Pazmiño Reyna Jaren Aaron
Robinson Feraud Steve William

Guayaquil-Ecuador

2026 - 2027

Table of Content

Introduction	3
Development	4
1. Repository URL	4
2. Feature List	4
3. Continuous Integration Workflow File (ci.yml)	5
4. Screenshot of a Passing Run on the Actions Tab	6
5. Screenshot of the Branch Protection Settings	7
6. Pull Requests Opened and Reviewed by Any Member	8
7. Screenshot of a Blocked Pull Request and Later Passing Run that Unblocked It	11
8. Decide Whether to Use Continuous Deployment	12
Questions	13
Conclusions	15
Recommendations	16
References	17

Introduction

Code integration in concurrent development teams often leads to bottlenecks when there are no automated validation mechanisms in place. This report details the transition from an analog management system to a digital CLI tool for the “CaféLibro” library. The core focus of the project lies in the implementation of a branch-based workflow, protected by automated tests through Continuous Integration (CI), ensuring that the main branch remains stable and functional at all times.

Development

1. Repository URL

<https://github.com/Arzel01/T7-Continuous-Integration>

2. Feature List

Feature	Assigned To
Register a member	José Adrian
Loan a book	Alex Otero
Return a book	Steve Robinson
List member currently has on loan	Bruno Romero
Report loans overdue	Jaren Pazmiño
Register a book in the catalogue	Jaren Pazmiño

3. Continuous Integration Workflow File (ci.yml)

```
name: ci

on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pylint pytest

      - name: Analysing the code with pylint
        run: |
          pylint $(git ls-files '*.py')

      - name: Run tests
        run: pytest -q

  release:
    needs: test
    if: github.event_name == 'push' && github.ref == 'refs/heads/master'
    runs-on: ubuntu-latest

    permissions:
      contents: write

    steps:
      - uses: actions/checkout@v4

      - name: Create version tag
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-actions[bot]@users.noreply.github.com"
          TAG_NAME="v$(date +%Y.%m.%d-%H%M%S)"
          git tag $TAG_NAME
          git push origin $TAG_NAME
```

4. Screenshot of a Passing Run on the Actions Tab

← ci

✓ Merge pull request #13 from Arzel01/feature/list-books #46

Summary

All jobs

- ✓ test
- ⌚ release

Run details

- ⌚ Usage
- 📄 Workflow file

Triggered via push 5 minutes ago

👤 Brun025-5 pushed -> 2d8968a master

Status: **Success**

Total duration: **17s**

Artifacts: —

ci.yml
on: push

✓ test 14s — release 0s

← ci

✓ Merge pull request #19 from Arzel01/feature/loans-overdue #61

Summary

All jobs

- ✓ test
- ✓ release

Run details

- ⌚ Usage
- 📄 Workflow file

Triggered via push 51 minutes ago

👤 Arzel01 pushed -> 67d1e8d master

Status: **Success**

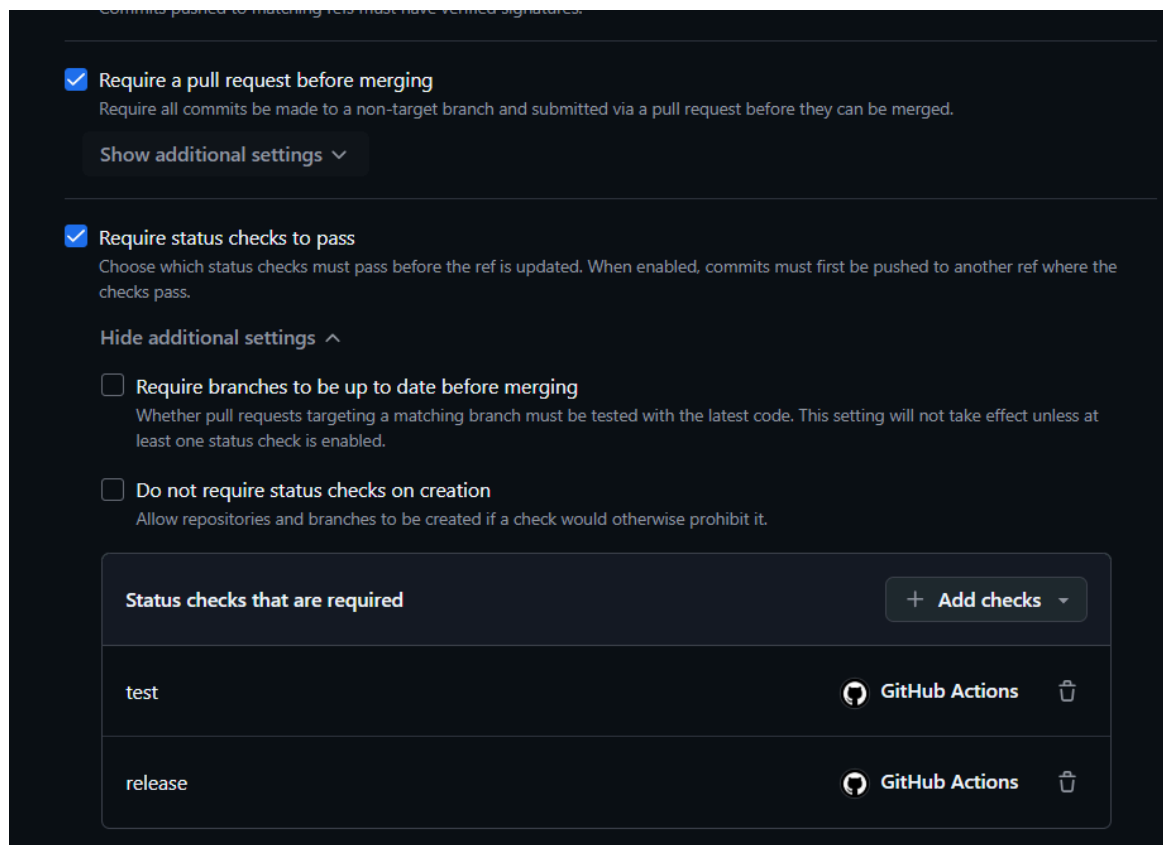
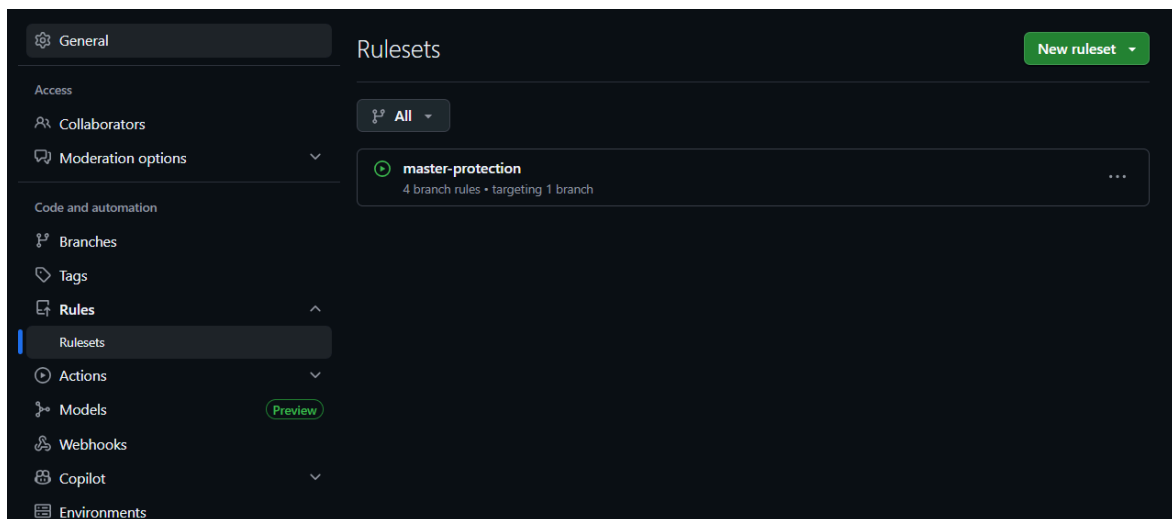
Total duration: **22s**

Artifacts: —

ci.yml
on: push

✓ test 12s — ✓ release 5s

5. Screenshot of the Branch Protection Settings



6. Pull Requests Opened and Reviewed by Any Member

José Adrián

The screenshot shows a GitHub pull request interface. At the top, the title is "fix: newline added #10" with an edit icon. Below the title, a status bar indicates "Merged" with a merge icon, followed by "Arzel01 merged 1 commit into master from feature/register" and a clock icon showing "14 minutes ago". Below this, a navigation bar shows "Conversation 0", "Commits 1", "Checks 2", and "Files changed 1". The main content area shows a comment by "Arzel01" (Owner) from "15 minutes ago" with the text "No description provided." and a sad face emoji. Below the comment is a commit history section showing a commit "fix: newline added" by "Arzel01" with a green checkmark and hash "bf30ba3". Below the commit is a merge summary showing "Arzel01 merged commit 1e93b24 into master" with "2 checks passed" and buttons for "View details" and "Revert". Below the merge summary is a message "Arzel01 deleted the feature/register branch 13 minutes ago" with a "Restore branch" button. At the bottom, a purple box with a merge icon contains the text "Pull request successfully merged and closed" and "You're all set — the branch has been merged." Below this box is a partially visible "Add a comment" input field.

Alex Otero

The screenshot shows a GitHub pull request interface. At the top, the title is "Feature/loan book #7" with an edit icon. Below the title, a status bar indicates "Merged" with a merge icon, followed by "alexoterol merged 2 commits into master from feature/loan-book" and a clock icon showing "26 minutes ago". Below this, a navigation bar shows "Conversation 0", "Commits 2", "Checks 2", and "Files changed 1". The main content area shows a comment by "alexoterol" (Collaborator) from "26 minutes ago" with the text "No description provided." and a sad face emoji. Below the comment is a commit history section showing a commit "fix: loan-book integration pylint issues" by "alexoterol" with a green checkmark and hash "f25635f". Below the commit is a merge summary showing "alexoterol merged commit 07e0d1e into master" with "26 minutes ago" and a "Revert" button. Below the merge summary is a message "Merge branch 'master' of https://github.com/Arzel01/t7-Continuous-Int..." with a red X and hash "c9fb126". At the bottom, a purple box with a merge icon contains the text "Pull request successfully merged and closed" and "You're all set — the feature/loan-book branch can be safely deleted." with a "Delete branch" button.

Jaren Pazmiño

Feature/register book #12

Merged JarenPOL1015 merged 2 commits into `master` from `feature/register-book` 4 minutes ago

Conversation 0 Commits 2 Checks 2 Files changed 2

JarenPOL1015 commented 5 minutes ago Collaborator

No description provided.

JarenPOL1015 added 2 commits 36 minutes ago

feat(catalogue): implement isolated book registration function e41933b

Merge branch 'master' of https://github.com/Arzel01/T7-Continuous-Int. de30e89

JarenPOL1015 merged commit 3584293 into `master` 4 minutes ago View details Revert

1 check failed

Pull request successfully merged and closed
You're all set — the `feature/register-book` branch can be safely deleted. Delete branch

Add a comment

Bruno Romero

feat: List the books a member currently has on loan and its own test. #13

Merged Brun025-5 merged 3 commits into `master` from `feature/list-books` now

Conversation 0 Commits 3 Checks 0 Files changed 2

Brun025-5 commented 42 minutes ago Collaborator

By adding this feature, system will be able to list books a member that has loaned recently. Moreover, it will be added a test for this feature.

feat: List the books a member currently has on loan and its own test. df8c00c

fix: incorrect variable name in test.py f94408b

fix: pylint fixure name 47464cb

Brun025-5 merged commit 2d8968a into `master` now View details Revert

2 checks passed

Pull request successfully merged and closed
You're all set — the `feature/list-books` branch can be safely deleted. Delete branch

Reviews
No reviews
Still in progress? [Convert to draft](#)

Assignees
No one—[assign yourself](#)

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request will update the `master` branch. [View commit](#)

Notifications

Steve Robinson

feature: Return a book, marking it available again. #5

 **Open** [stikrobinson](#) wants to merge 5 commits into [master](#) from [feature/return_book](#) 

 Conversation 0  Commits 5  Checks 2  Files changed 5

**stikrobinson** commented 36 minutes ago Collaborator 

No description provided.


 Mention @copilot in a comment to make changes to this pull request.

 **stikrobinson** added 5 commits 36 minutes ago

  [feature: Return a book, marking it available again.](#) ✖ 883dedf

  [fixture: pylint corrections in return_book](#) ✖ cc1b060

  [fixture: pylint test_return_book corrections](#) ✖ d9198b3

  [fixture: failed test correction](#) 307aead

  [Merge branch 'master' into feature/return_book](#) Verified ✖ 57d14b3



 **Some checks were not successful**
1 failing, 1 skipped checks 

1 failing check 

  [ci / test \(pull_request\)](#) Failing after 13s 

7. Screenshot of a Blocked Pull Request and Later Passing Run that Unblocked It

← ci

❌ Merge pull request #9 from Arzel01/feature/register #27 Re-run jobs

Summary

All jobs

test

release

Run details

Usage

Workflow file

Triggered via push 5 hours ago

Arzel01 pushed → 089df75 master

Status: Failure

Total duration: 15s

Artifacts: -

ci.yml

on: push

test 11s

release 0s



Summary

All jobs

test

release

Run details

Usage

Workflow file

test

failed 5 hours ago in 11s

Search logs

Explain error

- Set up job 0s
- Run actions/checkout@v4 1s
- Set up Python 3.12 0s
- Install dependencies 6s
- Analysing the code with pylint 2s
 - Run pylint \$ (git ls-files '*.py')
 - ***** Module test_library
 - test_library.py:41:0: C0304: Final newline missing (missing-final-newline)
 - *****
 - Your code has been rated at 9.82/10
 - Error: Process completed with exit code 16.
- Run tests 0s
- Post Set up Python 3.12 0s
- Post Run actions/checkout@v4 0s
- Complete job 0s



← ci

✅ Merge pull request #10 from Arzel01/feature/register #30 Re-run all jobs

Summary

All jobs

test

release

Run details

Usage

Workflow file

Triggered via push 5 hours ago

Arzel01 pushed → 1e93b24 master

Status: Success

Total duration: 19s

Artifacts: -

ci.yml

on: push

test 14s

release 0s



Summary

All jobs

test

release

Run details

Usage

Workflow file

Annotations

1 warning

test

succeeded 5 hours ago in 14s

Search logs

- Set up job 2s
- Run actions/checkout@v4 0s
- Set up Python 3.12 0s
- Install dependencies 6s
- Analysing the code with pylint 2s
- Run tests 1s
- Post Set up Python 3.12 0s
- Post Run actions/checkout@v4 0s
- Complete job 0s

8. Decide Whether to Use Continuous Deployment

What would you have to change in your workflow to make releases happen automatically, with no one approving them?

To transition to true Continuous Deployment, we would need to fundamentally alter how code moves from a developer's machine to production. First, we would remove the manual gating by modifying your GitHub branch protection rules to allow automated merging once the continuous integration pipeline completely passes, eliminating the requirement for human review approval. Second, we would update the `ci.yml` file to do more than just save a downloadable artifact. It would actively push and publish the packaged application (such as a Python wheel or npm tarball) directly to a central package registry or distribution server accessible by the librarians. Finally, we would integrate an automated versioning tool like `semantic-release` that reads your commit history, calculates the next version number based on your changes, and automatically updates the repository tags without human intervention.

If a bug got past your tests, it would reach users with no one checking first. Which of your features would be risky to release this way, and why?

If a bug managed to slip past your automated test suite, the features dealing with data modification and state validation would carry the most severe consequences. Specifically, Feature 2 (loaning a book) and Feature 3 (returning a book) are highly risky because they directly read and overwrite the shared JSON database. A silent bug in this logic could easily corrupt the integrity of the JSON file or fail to enforce critical business constraints, such as letting a single book be loaned to multiple people or wiping out existing records entirely, which would affect library operations.

Before trusting automatic releases, teams add safeguards: a way to undo a bad release, a copy of the system to test on first, and alarms when something fails in production. Research how at least one of these works and cite your source.

According to modern software architecture principles, an automated rollback strategy utilizes real-time log aggregators and metric monitors to track the health of an application immediately following a release [1]. If the newly deployed code triggers unexpected system crashes, unhandled errors, or bad data writes, predefined alarms are tripped. Once an alarm fires, the deployment pipeline automatically intervenes without human assistance, instantly reverting the active production environment back to the last known stable version to minimize the effects of the defective code [1].

State your decision: would you turn on continuous deployment for this application now, and what would have to be true for you to change your mind?

Based on the current state of the project, we would decide not to enable continuous deployment for this library application at this time. The primary blocker is architectural: because the application currently stores live library data (books, members, and active loans) in a local JSON file directly inside the project repository, any automated deployment runs a massive risk of overwriting or resetting the librarians' real-world data with the default template from Git. We would only change our decision if we migrate the data of the application to an external, persistent database independent of the code deployments.

Questions

Why is continuous integration worth the setup effort on a team project? What goes wrong on a shared repository without it?

Setting up Continuous Integration is worth the effort because it acts as an automated checklist to merge correctly in a team project, catching bugs and configuration mismatches within minutes of code being pushed rather than weeks later. Without CI, shared repositories inevitably descend into chaos: developers suffer from "works on my machine" syndrome due to hidden local dependencies, the main branch frequently breaks because untested code is merged blindly, and resolving massive merge conflicts on old branches turns into hard work. CI trades a small upfront setup investment for a permanently stable codebase, allowing the team to integrate code frequently, maintain styling consistency automatically, and refactor with total confidence.

This workshop used GitHub Actions. Research at least two other tools that do continuous integration and delivery (for example Jenkins, GitLab CI, CircleCI, Travis CI, or others). For each, say briefly how a project configures it and where it runs the build.

GitLab CI/CD

How it's configured: GitLab CI/CD is configured using a single YAML file named `.gitlab-ci.yml` placed directly in the root directory of the repository. Within this file, developers define the pipeline's structure, including distinct stages (such as build, test, and deploy), environment variables, and the specific shell scripts or commands that need to execute within each job.

Where the build runs: The builds are executed by an application called GitLab Runner. These runners can be hosted in two ways: they can run on GitLab's own cloud infrastructure (SaaS-hosted runners), or they can be self-hosted on a team's own local servers, virtual machines, or Docker containers for greater control over the build environment.

Jenkins

How it is configured: Jenkins projects are typically configured using a file named `Jenkinsfile`, which is stored in the source code repository. This file uses a Domain Specific Language (DSL) based on Groovy syntax, which can be written in either a Declarative format (a structured, easier-to-read layout) or a Scripted format (allowing advanced, flexible programmatic logic). Alternatively, legacy projects can also be configured manually using Jenkins' web-based user interface.

Where it runs the build: Jenkins follows a controller-agent architecture. The main Jenkins controller coordinates the pipeline, but the actual build steps are distributed to and executed on Jenkins Agents. These agents are dedicated physical servers, virtual machines, or dynamic Docker/Kubernetes containers provisioned specifically to run the build workloads.

Of the tools you compared, including GitHub Actions, which would you choose for the course project, and why? Consider where your code lives, the cost, and how much setup each one needs.

GitHub Actions is almost always the best choice.

- The practice requires a public GitHub repository for the team. Because GitHub Actions is natively integrated into the platform, there is no need to connect external integrations, which would be required if using Jenkins or GitLab CI.
- GitHub Actions is completely free for public repositories, offering unlimited build minutes. This perfectly matches the project requirement to use a public team repository without incurring infrastructure costs.
- The workshop document shows that setting up GitHub Actions only requires dropping a single `ci.yml` file into the `.github/workflows/` directory. It runs automatically on GitHub-hosted runners immediately. In contrast, Jenkins demands significant configuration, including server installation, which is excessively complex for a short workshop.

How does continuous integration fit with the way agile teams work in short iterations?

Continuous integration fits perfectly with an Agile methodology because it provides the rapid feedback loop necessary to sustain a high development velocity without sacrificing software quality. By automatically building and testing code in small increments on every push, CI eliminates late integration issues where combining weeks of isolated work results in complex defects. This automated checks ensures that conflicts are detected and repaired immediately when the cost to fix them is lowest, keeping the main branch in a permanently stable, verified, and potentially shippable state at the end of every sprint.

Conclusions

Implementing Continuous Integration (CI) through GitHub Actions successfully eliminated the risk of "late integration" bottlenecks and code conflicts during the concurrent development of the CaféLibro application. By combining strict branch protection rules with automated pylint and pytest checks, the team established an automated gatekeeper that intercepted and blocked code errors from corrupting the mainline codebase. Ultimately, while the pipeline reached the maturity to automate version tagging, the team correctly determined that moving to Continuous Deployment is currently restricted by the application's local JSON database architecture, which requires human oversight to prevent live data from being accidentally overwritten or lost.

Recommendations

To advance the setup of the CaféLibro project, it is highly recommended to migrate the application's state storage away from a local JSON file in the repository and integrate an external, persistent database instead. Decoupling live user data from the codebase would eliminate the high structural risk of data corruption or data loss during deployment cycles, clearing the primary bottleneck that prevents the safe adoption of Continuous Deployment. Additionally, the team should expand the pipeline's automated test coverage to include comprehensive integration tests for data-modifying functions, such as book loans and returns, ensuring that critical business constraints are fully protected by the CI gatekeeper before any manual or automated delivery takes place.

References

- [1] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Boston, MA, USA: Addison-Wesley, 2015.
- [2] GitLab, "GitLab CI/CD architecture," *GitLab Documentation*, 2026. [Online]. Available: <https://docs.gitlab.com/ee/ci/>
- [3] CircleCI, "CircleCI configuration reference," *CircleCI Docs*, 2026. [Online]. Available: <https://circleci.com/docs/configuration-reference/>
- [4] P. M. Duvall, S. Matyas, and A. Glover, *Continuous Integration: Improving Software Quality and Reducing Risk*. Boston, MA, USA: Addison-Wesley, 2007.
- [5] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.
- [6] N. Pathania, *Learning Continuous Integration with Jenkins: An end-to-end guide to creating operational, secure, resilient, and cost-effective CI/CD processes*, 3rd ed. Birmingham, UK: Packt Publishing, 2024.